



Prompting Strategies for Network Protocol Generation: An Ablation Study on Flooding and Neighbor Discovery in TinyOS

Mohammed Amir Khabaza

University of California, Merced | CSE 188: Natural Language Processing



The Problem & Excitement (Innovation)

- The Digital Divide: LLMs excel at high-level scripting (Python) but fail at hardware-centric languages like nesC, which represent a statistically insignificant fraction of their training data.
- The Concurrency Challenge: TinyOS uses an asynchronous Split-Phase Execution model. Zero-shot LLMs frequently hallucinate synchronous, "blocking" logic, causing total system failures.

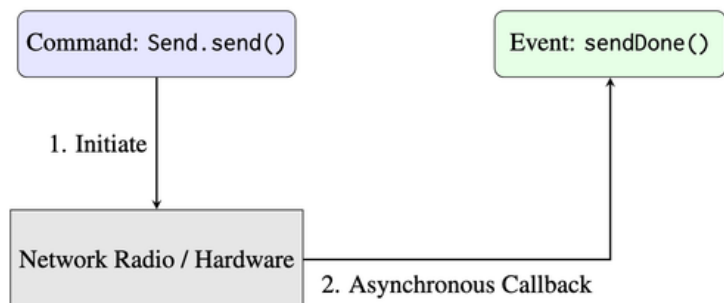


Figure 1: The Split-Phase Execution Model in TinyOS. Zero-shot LLMs struggle to link asynchronous callbacks without explicit logical mapping.

Methodology (Ablation Study)

- **Zero-Shot Baseline:** Direct protocol description without guided reasoning.
- **Chain-of-Thought (CoT):** Forced explicit logical breakdowns (e.g., defining nx_struct payloads, identifying TinyOS interfaces, mapping state cache).
- **Target Protocols:** Flooding and Neighbor Discovery.

Soundness (Rigor & Hardware Metrics)

Quantitative Results: The Power of CoT

- **Compilation:** GPT-4o compilation success jumped from 55% (Zero-Shot) to 85% (CoT).
- **Simulation (TOSSIM):** Logical soundness (delivering packets without broadcast storms) improved by 35% in GPT-4o.

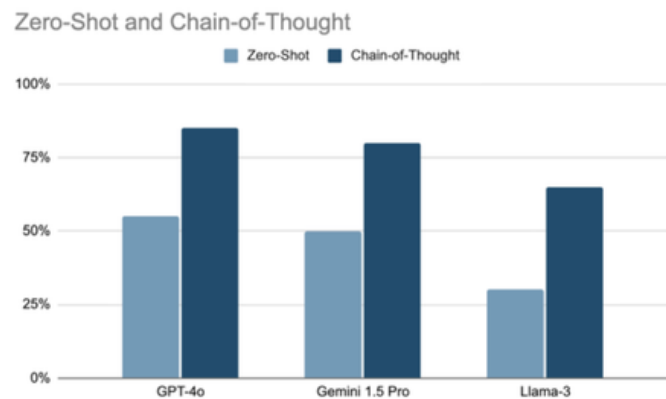


Figure 2: Compilation Success Rate: A comparison of Zero-Shot versus Chain-of-Thought prompting across models.

Hardware Resource Benchmarking

Can LLM code actually fit on a sensor node? TinyOS nodes (TelosB) have only 10KB of RAM.

Model (CoT)	SRAM (B)	Flash (B)	Overhead
Baseline (Manual)	752	12,440	–
GPT-4o	842	13,912	12.0%
Gemini 1.5 Pro	896	14,208	19.1%
Llama-3 (70B)	912	14,880	21.3%

Table 1: Memory footprint comparison of generated Flooding protocols on the TelosB platform.

Error Analysis & Case Studies (Deep Soundness)

Case Study: The Broadcast Storm

- **Failure:** Zero-shot models (e.g., Llama-3) omitted the sequence number cache, causing infinite routing loops and crashing the simulator.
- **Success:** CoT forced the model to allocate static memory for caching, successfully dropping duplicate packets.

```
event message_t* Receive.receive(
  message_t* msg, void* payload,
  uint8_t len) {
  // FATAL ERROR: No sequence number
  // cache check.
  // Node will infinitely forward the
  // same packet.
  call AMSend.send(AM_BROADCAST_ADDR,
    msg, len);
  return msg;
}
```

Listing 1: Zero-Shot Generation: Naive forwarding resulting in a broadcast storm.

```
uint16_t seq_cache[CACHE_SIZE];

event message_t* Receive.receive(
  message_t* msg, void* payload,
  uint8_t len) {
  flooding_msg_t* fmsg = (
    flooding_msg_t*)payload;

  if (checkCache(fmsg->src, fmsg->seq))
  {
    return msg; // Drop duplicate
    packet safely
  }

  updateCache(fmsg->src, fmsg->seq);
  call AMSend.send(AM_BROADCAST_ADDR,
    msg, len);
  return msg;
}
```

Listing 2: Chain-of-Thought Generation: Logically sound caching mitigating routing loops.

A Taxonomy of Architectural Failures

We identified three novel failure modes when applying LLMs to embedded systems:

- **The Interface Mirage:** Hallucinating interface usage without proper .nc wiring.
- **Synchronous Hallucination:** Forcing blocking logic into asynchronous handlers.
- **State-Space Fragmentation:** Attempting illegal dynamic memory allocation (malloc).

Failure Taxonomy & Future Work

- **Key Failure Modes:**
 - * **Interface Mirage:** Using interfaces without proper wiring.
 - **Synchronous Hallucination:** Attempting to force blocking logic into asynchronous handlers.
- **Future RAG Pipeline:** We propose a **Retrieval-Augmented Generation (RAG)** architecture using a vector database of TinyOS APIs to "re-ground" models in hardware constraints.

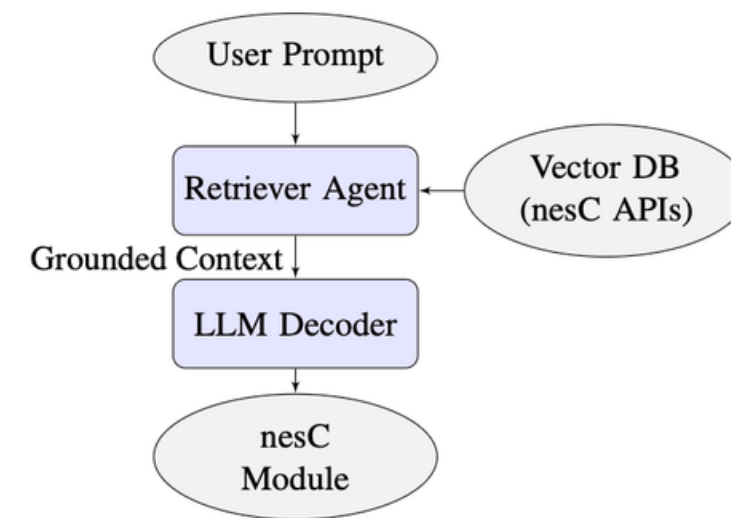


Figure 4: Proposed RAG Architecture for Grounded Protocol Generation.